

Hive DDL

Цель темы

Научиться накрывать разные типы данных внешними и управляемыми таблицами.

Задачи темы

- ✓ Создать таблицу, вставить строки и удалить таблицу с данными
- ✓ Создать внешнюю таблицу над существующими данными
- ✓ Создать партиционированную таблицу, посмотреть, как лежат данные
- ✓ Создать бакетированную таблицу, понять, как в ней распределяются данные

Функции Hive

- 1 DDL: TABLE (Internal / External), VIEW
- 2 Составные типы данных
- 3 Partitioning, Bucketing
- 4 Различные опции загрузки данных LOAD / INSERT
- 5 Pluggable Input / Output format
- 6 Pluggable SerDe libs

Hive не может

- 1 DELETE row in TABLE
- 2 UPDATE row in TABLE
- 3 MERGE row in TABLE
- 4 SCHEMA on WRITE
- 5 Primary and Foreign Key

Простые типы данных в Hive

Numeric Types:

TINYINT: 1-byte signed integer

SMALLINT: 2-byte signed integer

INT: 4-byte signed integer

BIGINT: 8-byte signed integer

FLOAT: Single-precision floating-point number

DOUBLE: Double-precision floating-point number

DECIMAL: Fixed-point decimal number

String Types:

STRING: Variable-length character string

CHAR: Fixed-length character string

VARCHAR: Variable-length character string

Date and Time Types:

DATE: Represents a date (year, month, and day)

TIMESTAMP: Represents a date and time

INTERVAL: Represents a time interval

Boolean Type:

BOOLEAN: Represents a boolean value (true or false)

Комплексные типы данных в Hive

ARRAY: Ordered collection of elements

MAP: Collection of key-value pairs

STRUCT: Collection of named fields

Managed Table

- 1 Данные хранятся в директории hive / warehouse на HDFS
- 2 Фиксирует схему (именованных столбцов, типов данных)
- 3 Кладёт схему + статистику (мету) в метастор
- 4 Загрузка данных INSERT / LOAD

Пример

ARRAY: Ordered collection of elements

MAP: Collection of key-value pairs

STRUCT: Collection of named fields

Создание таблицы

```
CREATE TABLE marketplace_clients (  
  client_id STRING,  
  product_ids ARRAY<STRING>,  
  demographic STRUCT<gender:STRING, age:INT>,  
  purchase MAP<STRING, INT>,  
  preferences MAP<STRING, ARRAY<STRING>>,  
  orders ARRAY<STRUCT<gender:STRING, age:INT>>  
)  
ROW FORMAT DELIMITED  
  FIELDS TERMINATED BY '|'   
  COLLECTION ITEMS TERMINATED BY ','  
  MAP KEYS TERMINATED BY ':'  
STORED AS TEXTFILE;
```

- 1 Пример создания managed-таблицы с комплексными типами
- 2 Данные будут сохранены в отдельной директории
user/hive/warehouse

Форматы файлов в таблице

- 1 TEXT
- 2 JSON
- 3 AVRO
- 4 ORC — default-формат
- 5 PARQUET
- 6 XML

Проверка таблицы

DESC marketplace_clients; — проверка типов колонок.

SHOW CREATE TABLE marketplace_clients; — загрузка DDL.

Наполнение таблицы insert

INSERT INTO marketplace_clients

VALUES ('1001',
 array('p1', 'p2', 'p3'),
 named_struct('gender', 'male', 'age', 25),
 map('product1', 5, 'product2', 2),
 map('category1', *array*('electronics'),
 array(*named_struct*('gender',
 'female',
 'age',
 30)))));

'category2', *array*('books', 'furniture'))),

INSERT OVERWRITE INTO marketplace_clients

VALUES (.....

Наполнение таблицы load из txt-файлов

```
LOAD DATA INPATH '/user/stage/employee.txt' OVERWRITE INTO TABLE marketplace_clients;
```

```
LOAD DATA LOCAL INPATH '/user/stage/employee.txt' OVERWRITE INTO TABLE marketplace_clients;
```

Удаление таблицы вместе с данными

```
DROP TABLE marketplace_clients;
```

Данные в папке будут удалены

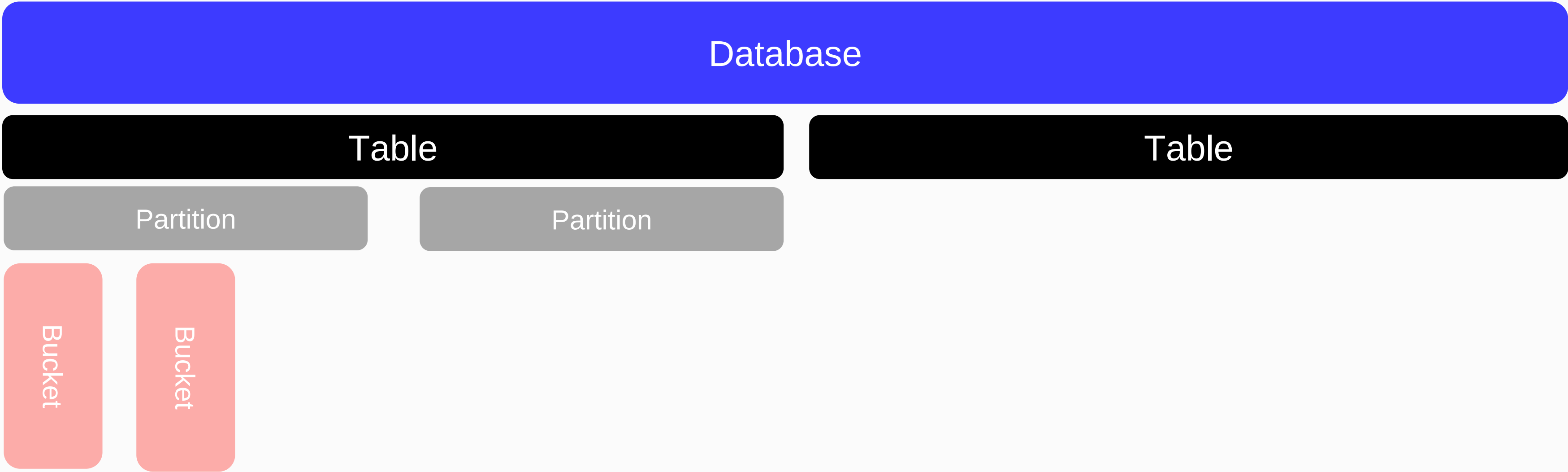
```
'/user/hive/warehouse/marketplace_clients/000000_0'
```

External table

```
LOAD DATA INPATH '/user/stage/employee.txt' OVERWRITE INTO TABLE marketplace_clients;
```

```
LOAD DATA LOCAL INPATH '/user/stage/employee.txt' OVERWRITE INTO TABLE marketplace_clients;
```

Модель данных Hive



Partitioning

- 1 Данные разложены по директориям
- 2 Горизонтальное разделение
- 3 1+ колонок
- 4 Запросы, использующие партиции, будут быстрее
- 5 Как решить, по каким колонкам делить

Создание External Partitioning table

```
CREATE EXTERNAL TABLE clients_partitioned (  
    name STRING,  
    products_id ARRAY<STRING>,  
    gender_age STRUCT<gender:STRING, age:INT>,  
    bonus_score MAP<STRING, INT>,  
    title MAP<STRING, ARRAY<STRING>>  
)  
PARTITIONED BY (Year INT, Month INT)  
STORED AS ORC  
LOCATION '/user/vopolski/hive-features/employee_partitioned';
```

Вложенные партиции

```
CREATE EXTERNAL TABLE clients_partitioned (  
  name STRING,  
  products_id ARRAY<STRING>,  
  gender_age STRUCT<gender:STRING, age:INT>,  
  bonus_score MAP<STRING, INT>,  
  title MAP<STRING, ARRAY<STRING>>  
)  
  PARTITIONED BY (Year INT, Month INT)  
  STORED AS ORC  
  LOCATION '/user/vopolski/hive-features/employee_partitioned';
```

Вложенные партиции

```
CREATE EXTERNAL TABLE clients partitioned (  
  name STRING,  
  products_id ARRAY<STRING>,  
  gender_age STRUCT<gender:STRING, age:INT>,  
  bonus_score MAP<STRING, INT>,  
  title MAP<STRING, ARRAY<STRING>>  
)  
PARTITIONED BY (Year INT, Month INT)  
STORED AS ORC  
LOCATION '/user/vopolski/hive-features/employee_partitioned';
```

Лучше использовать дату, а не год и месяц

```
CREATE EXTERNAL TABLE clients_partitioned (  
  name STRING,  
  products_id ARRAY<STRING>,  
  gender_age STRUCT<gender:STRING, age:INT>,  
  bonus_score MAP<STRING, INT>,  
  title MAP<STRING, ARRAY<STRING>>  
)  
PARTITIONED BY (Date STRING)  
STORED AS ORC  
LOCATION '/user/raw/employee_partitioned';
```

Лучше использовать дату, а не год и месяц

```
SELECT *  
FROM clients_partitioned  
WHERE  
    (year = 2019 AND month = 1 AND day = 1) OR  
    (year = 2019 AND month = 1 AND day = 2);
```

```
SELECT *  
FROM clients_partitioned  
WHERE Date in ('2019-01-01', '2019-01-02');
```

```
SELECT *  
FROM clients_partitioned  
WHERE month = 1;  
-- Performance issue
```

Static Partitioning

```
INSERT OVERWRITE TABLE clients_partitioned
  PARTITION (city = 'Montreal')
SELECT name,
         products_id,
         gender_age
FROM clients
WHERE city = 'Montreal';
```

Static Partitioning

INSERT OVERWRITE TABLE clients_partitioned

PARTITION (city = 'Montreal')

SELECT name,

products_id,

gender_age

FROM clients

WHERE city = 'Montreal';

Dynamic Partitioning

```
SET hive.exec.dynamic.partition=true;
```

```
SET hive.exec.dynamic.partition.mode=nostrict;
```

```
SET hive.max.dynamic.partitions=1000;
```

```
INSERT OVERWRITE TABLE clients_partitioned
```

```
  PARTITION (city)
```

```
SELECT
```

```
  name,
```

```
  products_id,
```

```
  gender_age
```

```
FROM clients;
```

Рекомендации

- 1 Вложенные партиции не всегда эффективны
- 2 Много партиций забивают Metastore
- 3 HDFS не любит мелкие файлы
- 4 Используйте партиции, если:
 - данных очень много
 - в запросах есть фильтрация или джойн по колонке
- 5 Колонки с небольшим количеством уникальных значений

Что если много уникальных значений?

Например, колонка clients_id:

1

2

3

4

5

6

7

8

9

10

Bucketing

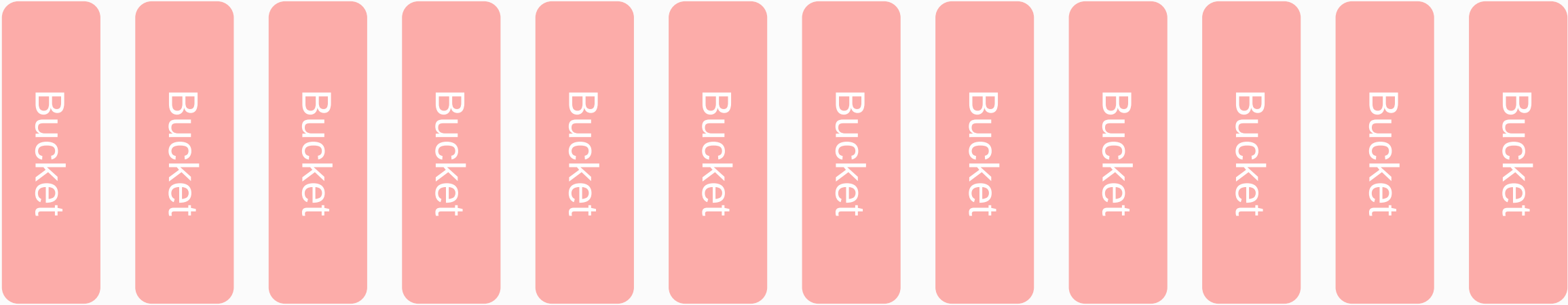
- 1 Делит данные вертикально с использованием хеш-функции
- 2 Указывается количество buckets
- 3 Особенно полезен при использовании Joins

```
CREATE TABLE clients_bucketed (  
  client_id STRING,  
  name      STRING,  
  products_id ARRAY<STRING>,  
  gender_age STRUCT<gender:STRING, age:INT>,  
  bonus_score MAP<STRING, INT>,  
  title      MAP<STRING, ARRAY<STRING>>,  
  city       STRING  
) CLUSTERED BY (client_id) INTO 10 BUCKETS  
  STORED AS ORC;
```

Наполнение bucketed table

Необходимо явно указывать в настройках:

```
SET hive.enforce.bucketing=true;  
SET hive.exec.dynamic.partition=true;  
INSERT INTO TABLE clients_bucketed  
SELECT * FROM clients;
```

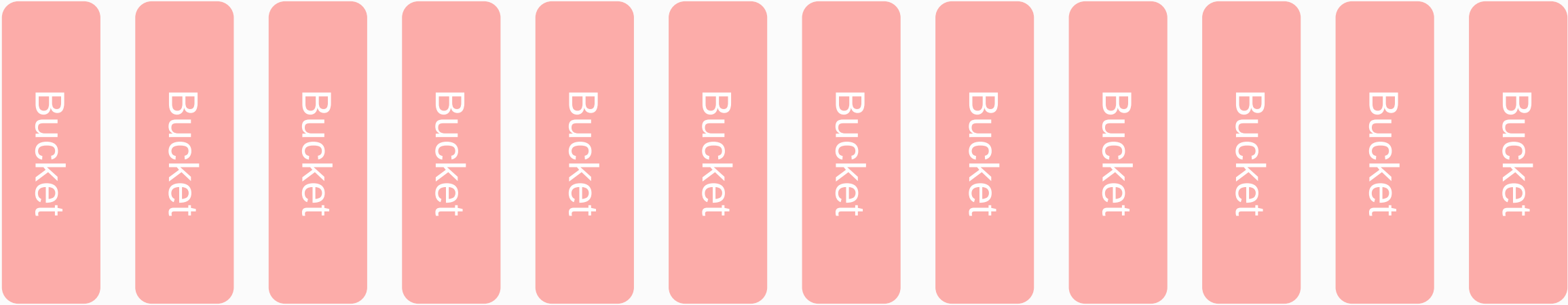
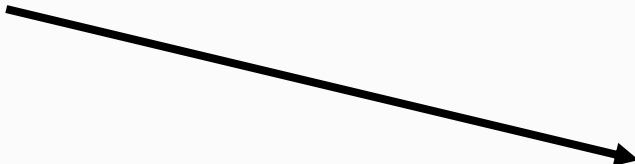


Наполнение bucketed table

Id = 1, 2, 3, 4, 5, 6, 7, 8, 9,, 1024, 1025



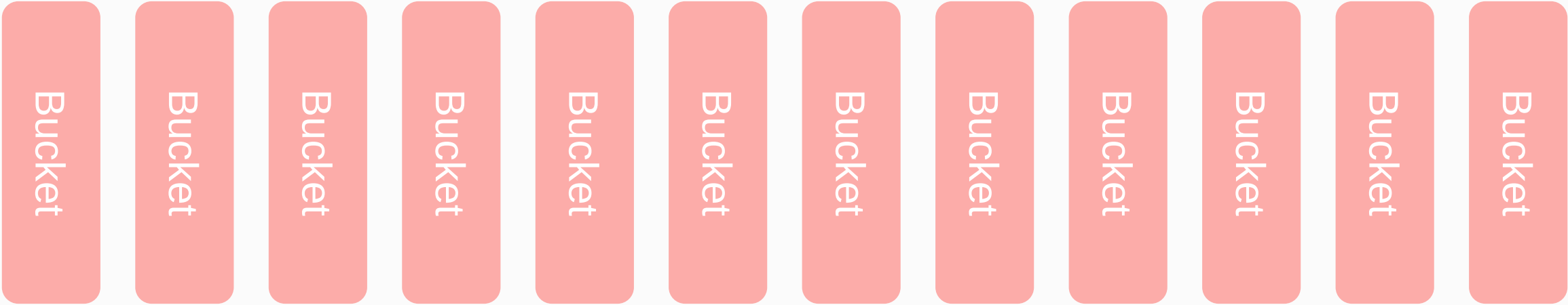
Hash(id) % 12 = 0 ~ 11



Наполнение bucketed table

Id = 1, 2, 3, 4, 5, 6, 7, 8, 9,, 1024, 1025

Hash(id) % 12 = 0 ~ 11



Выводы темы

- ✓ Hive поддерживает managed table
- ✓ Hive позволяет накрывать данный external table
- ✓ Партиционирование полезно по хронологическим колонкам с небольшим количеством значений
- ✓ Bucketing полезен для таблиц, участвующих в соединении по одному ключу